SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME:Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

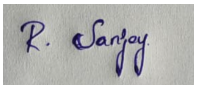
Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I Sanjay Raghavendra/ 192411041 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

ANSWERS :

1. Reverse Engineering an ATM Withdrawal Module Using Black-Box Testing

Introduction

An ATM withdrawal module allows users to withdraw money from their bank accounts based on withdrawal limits, available balance, and account validation rules. In this scenario, only the input-output behavior is known, and the original test documentation is unavailable. Reverse engineering can be used to infer the likely testing techniques and reconstruct an improved test suite.

System Analysis and Functional Decomposition

The ATM withdrawal module can be divided into the following functional components:

1. Input Validation: Checks whether the withdrawal amount is numeric, positive, and correctly formatted.
2. Account Validation: Verifies that the account exists and is active.
3. Balance Validation: Checks whether the available balance is sufficient.
4. Withdrawal Limit Validation: Ensures that the requested amount is within the minimum and maximum withdrawal limits.
5. Transaction Processing: Deducts the approved amount from the account balance.

6. Error Handling: Displays appropriate messages when a withdrawal request is invalid or unsuccessful.

Identification of Equivalence Classes

Equivalence Class Partitioning (ECP) divides inputs into groups expected to produce similar behavior.

For the withdrawal amount, the following classes can be inferred:

- Negative withdrawal amount: invalid.
- Zero withdrawal amount: invalid.
- Amount below the permitted minimum: invalid.
- Amount within the permitted limits and supported denomination: potentially valid.
- Amount above the maximum withdrawal limit: invalid.
- Amount greater than the available balance: invalid.
- Blank or non-numeric amount: invalid.

These classes allow representative values to be selected instead of testing every possible amount.

Application of Boundary Value Analysis

Boundary Value Analysis (BVA) focuses on values around the permitted limits.

Assume the minimum withdrawal amount is L and the maximum is U. Test $L - 1$, L, and $L + 1$, as well as $U - 1$, U, and $U + 1$, using the smallest supported currency unit.

Also test the exact available balance and amounts immediately below and above it. If ATM denomination restrictions apply, verify that valid amounts conform to the supported banknote denominations.

The actual withdrawal limits must be obtained from the system requirements.

Reconstructed Test Cases

Test Case 1: Enter a valid amount within the withdrawal limits and below the available balance. The system should approve the withdrawal and deduct the correct amount.

Test Case 2: Enter an amount below the minimum limit. The system should reject the request and display an appropriate error.

Test Case 3: Enter an amount equal to the minimum limit. The system should follow the specified boundary rule.

Test Case 4: Enter an amount equal to the maximum limit. The system should accept it if all other conditions are satisfied.

Test Case 5: Enter an amount above the maximum limit. The system should reject the request.

Test Case 6: Enter an amount greater than the available balance. The system should reject the withdrawal without changing the balance.

Test Case 7: Enter zero, a negative amount, a blank value, or non-numeric input. The system should reject the invalid input.

Test Case 8: Simulate a failure during cash dispensing or transaction processing. The system should follow the specified recovery procedure and prevent an incorrect final balance.

Evaluation and Improved Strategy

The observed behavior suggests that the original testing may have included ordinary input validation and some boundary tests. However, without test documentation, the exact original techniques cannot be confirmed.

The improved strategy should combine ECP, BVA, negative testing, transaction-state testing, and regression testing. Each test should verify the displayed message, account balance, transaction record, and cash-dispensing outcome.

Conclusion

Reverse engineering helps reconstruct the ATM's expected validation and transaction behavior from observable results. Combining ECP and BVA with transaction and failure scenarios improves test completeness and reduces the risk of incorrect withdrawals.

2. Reverse Engineering a Software Module Using Control Flow Graphs and Path Testing

Introduction

When execution traces and code snippets are available but test documentation is missing, the testing strategy can be reconstructed by examining the program's control flow. A Control Flow Graph (CFG) represents the order in which statements and decisions may execute.

System Decomposition

The software module can be divided into:

1. Input initialization.
2. Input validation.
3. Conditional decision processing.
4. Loop execution.
5. Result calculation.
6. Error handling.
7. Program termination.

The actual components depend on the supplied code and execution traces.

Construction of a Conceptual CFG

Consider the following illustrative logic:

```
Start
  |
Read input
  |
Is input valid?
  | Yes            | No
Process input     Display error
  |                |
Is more data?       End
  | Yes    | No
Repeat    Display result
             |
            End
```

This representation contains a decision for input validation and another decision controlling repetition.

The diagram is illustrative because the assessment question does not provide a specific code snippet or execution trace.

Identification of Decision Points and Loops

A decision point creates alternative execution paths. Each decision should have its True and False outcomes tested.

Loops introduce repeated execution and require tests for termination. Depending on the code, useful loop scenarios include zero iterations, one iteration, multiple iterations, and the maximum permitted number of iterations.

Execution traces can be compared against the CFG to identify branches or paths that have not been executed.

Evaluation of Coverage Criteria

Statement coverage: Every executable statement is executed at least once.

Branch coverage: Every decision outcome is executed at least once.

Path coverage: Different sequences of control-flow branches are exercised.

Statement coverage may miss untested False branches. Branch coverage is stronger, but it does not guarantee that every possible sequence of decisions has been tested.

Basis-Path Testing

Cyclomatic complexity estimates the number of linearly independent paths in a control flow graph:

\[
V(G)=E-N+2P
\]

Where E represents edges, N represents nodes, and P represents connected components.

For a single connected graph:

\[
V(G)=E-N+2
\]

The calculated value guides the selection of independent basis paths. A test case should be designed to execute each identified path.

Reconstructed Testing Strategy

The improved test suite should include:

- Valid input execution.
- Invalid input execution.
- True and False outcomes of each decision.
- Zero-iteration and repeated-loop scenarios where applicable.
- Loop termination conditions.
- Error-handling paths.
- Tests for exceptional or unexpected inputs.

The CFG should be reviewed against the actual source code, and test cases should be linked to the corresponding branches.

Conclusion

CFG reconstruction and basis-path testing help recover the likely white-box testing approach from code and execution traces. By identifying independent paths, branches, and loops, the testing team can improve coverage and detect defects that statement-only testing may miss.

3. Reverse Engineering a Web Application Login and Transaction System

Introduction

A web application contains login and transaction functionality, but the original testing techniques are unknown. Only defect logs and test results are available. By analyzing defect patterns, the testing team can infer likely testing methods and identify missing coverage.

System Decomposition

The system can be divided into the following components:

1. Login Input Validation: Checks username and password formats.
2. Authentication: Verifies credentials.
3. Session Management: Maintains authenticated user sessions.
4. Authorization: Checks whether the user can perform a requested transaction.
5. Transaction Validation: Validates transaction amounts and account conditions.
6. Transaction Processing: Executes the transaction.
7. Logging and Error Handling: Records results and handles unsuccessful operations.

Reverse Engineering the Existing Approach

If defects occur at minimum or maximum transaction amounts, the original testing may have inadequately applied Boundary Value Analysis.

If invalid credentials or malformed inputs cause failures, equivalence classes may not have been sufficiently

covered.
If defects occur only after particular sequences of login and transaction operations, the original tests may have missed important control-flow paths or session-state transitions.
These are reasonable inferences from defect patterns, not proof of the exact original testing methods.
Application of Equivalence Class Partitioning
For login testing, divide inputs into valid credentials, invalid credentials, missing credentials, and malformed credentials.
For transaction testing, divide amounts into invalid negative values, zero values, permitted amounts, amounts exceeding transaction limits, and amounts exceeding available balances.
Test representative values from each class.
Application of Boundary Value Analysis
For transaction limits, test values immediately below, equal to, and above the permitted minimum and maximum.
For login fields, test empty values, minimum permitted lengths, maximum permitted lengths, and values just outside those limits.
The actual constraints must follow the application's requirements.
Application of Path Testing
Construct a CFG for the login and transaction process.
A conceptual path is:
Start → Validate Login Input → Verify Credentials → Create Session → Check Authorization → Validate Transaction → Execute Transaction → Record Result → End.
Alternative paths should cover invalid input, incorrect credentials, session expiration, unauthorized transactions, insufficient balance, and transaction-processing failures.
Use basis-path testing to identify independent paths and branch coverage to verify the True and False outcomes of decisions.
Improved Hybrid Testing Strategy
Combine ECP and BVA with CFG analysis and path testing. Test successful login, failed login, expired sessions, unauthorized transactions, boundary amounts, and transaction failures.
For each test, verify the response, session state, authorization result, account balance, and transaction log. Add regression tests for every defect discovered.
Conclusion
Reverse engineering defect patterns helps identify likely gaps in the existing test strategy. Integrating ECP, BVA, branch coverage, and path testing provides broader coverage of both external behavior and internal control flow.

4. Reverse Engineering an Inventory Management System Using Transaction Logs
Introduction
A legacy inventory management system provides successful and failed transaction logs for stock updates and low-stock alerts. By examining these records, the testing team can infer stock-validation rules, boundary conditions, and missing input classes.
System Decomposition
The inventory update process can be divided into:
1. Product identification.
2. Current-stock retrieval.
3. Quantity validation.
4. Stock update calculation.

5. Database update.
6. Low-stock threshold evaluation.
7. Alert generation.
8. Transaction logging and error handling.

Analysis of Transaction Logs

Successful transactions indicate combinations of inputs that the system accepts. Failed transactions help identify rejected values and possible validation rules.

For example, a failed update involving a negative quantity may indicate that negative stock changes are prohibited. A low-stock alert appearing at a particular stock level may indicate a threshold condition.

However, logs alone cannot always reveal the exact business rule. The inferred rules must be confirmed against requirements or additional controlled tests.

Identification of Equivalence Classes

For stock-update quantities, possible classes include:

- Negative quantity.
- Zero quantity.
- Valid positive quantity.
- Quantity exceeding a configured limit.
- Non-numeric or missing quantity.
- Quantity causing stock to exceed capacity.
- Quantity causing insufficient stock during a removal operation.

For resulting stock levels, classify the stock as negative or invalid, exactly at the low-stock threshold, below the threshold, above the threshold, or at maximum capacity where applicable.

Application of Boundary Value Analysis

Assume the low-stock threshold is T. Test resulting stock values of $T - 1$, T, and $T + 1$.

If stock cannot become negative, test removal quantities that leave stock at zero, one unit above zero, or below zero.

If a maximum stock capacity C exists, test resulting quantities of $C - 1$, C, and $C + 1$.

These values help reveal off-by-one errors in update and alert logic.

Reconstructed Test Cases

Test Case 1: Increase stock using a valid positive quantity. Verify the resulting stock.

Test Case 2: Remove stock while sufficient stock exists. Verify the correct deduction.

Test Case 3: Attempt to remove more stock than is available. Verify that the operation follows the defined rejection or reservation rule.

Test Case 4: Update stock so that the resulting quantity equals the low-stock threshold. Verify the alert behavior.

Test Case 5: Update stock so that it falls below the threshold. Verify that the alert is generated if required.

Test Case 6: Enter a negative or non-numeric quantity. Verify that the input is rejected.

Test Case 7: Simulate a database failure during an update. Verify that stock and transaction records remain consistent.

Conclusion

Reverse engineering transaction logs helps infer hidden inventory rules and identify missing input classes. Combining ECP and BVA with database-failure testing improves stock-update accuracy and ensures that low-stock alerts operate according to the specified requirements.

5. Reverse Engineering a Library Management Module from CPU and Memory Profiles

Introduction

A compiled executable file and its CPU and memory profiles are available for a library management module. The module experiences resource spikes during stress testing. These observations can help identify resource-intensive operations and guide the reconstruction of white-box testing requirements.

System Decomposition

The library management module can be divided into:

1. Book search and retrieval.
2. Member validation.
3. Book issue processing.
4. Book return processing.
5. Fine calculation.
6. Database access.
7. Repeated record processing.
8. Memory allocation and cleanup.

CPU and memory profiles may help identify expensive operations, but they do not reveal the exact source-code structure by themselves.

Analysis of Execution Profiles

A sustained CPU spike may indicate repeated computations, inefficient searches, nested loops, or frequent database queries.

A sudden memory increase may indicate large collections, repeated object allocation, cached records, or memory that is not released properly.

These are possible explanations that should be investigated through profiling, controlled input changes, and available debugging tools.

Construction of a Conceptual CFG

A conceptual control flow graph for book issuing may be represented as:

Start → Read Request → Validate Member → Is Member Valid?

- If No, return an error.
- If Yes, check book availability.
- If the book is unavailable, return an availability message.
- If available, check issue limits.
- If the member is within the limit, issue the book and update the database.
- Otherwise, reject the request.
- Record the result and terminate.

Loops may occur when searching book records or processing multiple requests.

Reconstructed Path Testing Strategy

Identify each decision and construct tests for both outcomes. For resource-intensive operations, test:

- Empty or very small datasets.
- Typical library datasets.
- Large datasets.
- Maximum supported request batches.
- Missing or invalid records.
- Repeated searches and repeated issue/return operations.
- Database failures and unsuccessful transactions.

Use CFG analysis and basis-path testing to cover independent execution paths. Correlate each test with CPU usage, memory consumption, execution time, and output correctness.

Evaluation of System Stability

Stress tests should gradually increase the number of records and concurrent requests. Monitor whether CPU usage stabilizes, memory returns to an acceptable level, and response times remain within requirements.

A resource spike should not automatically be treated as proof of a loop defect. Profiling evidence and repeatable test cases are needed to identify the cause.

Conclusion

Execution profiles provide clues about resource-intensive paths but cannot fully reconstruct source code. Combining conceptual CFG analysis, path testing, and controlled stress testing helps identify performance bottlenecks and improves library management system stability.

6. Reverse Engineering a Mobile Payment Gateway Using State Transition Testing

Introduction

A mobile payment gateway produces API responses for Pending, Success, Reversal, and Timeout states. By examining the response sequence and transaction data, the testing team can reconstruct the underlying state machine and identify legal and illegal transitions.

System Decomposition

The payment gateway can be divided into:

1. Transaction request validation.
2. Payment initiation.
3. Transaction status management.
4. Payment confirmation.
5. Timeout handling.
6. Reversal processing.
7. API response generation.
8. Transaction logging and reconciliation.

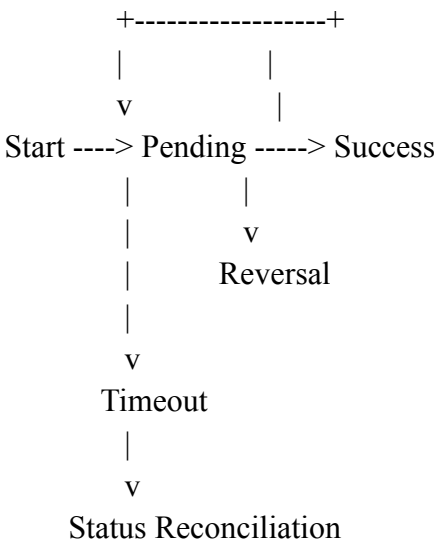
Identification of the State Machine

The four observed states are:

- Pending: The payment request has been initiated, but the final result is not yet confirmed.
- Success: The payment has completed successfully.
- Reversal: A previously authorized or completed payment is being reversed according to the payment rules.
- Timeout: A response was not received within the permitted time.

The exact meaning of Reversal and Timeout must be verified from the API specification. In particular, a timeout does not necessarily mean that the payment failed; the final status may require reconciliation.

Conceptual State-Transition Diagram



This is a conceptual model. The permitted transitions and recovery behavior must be confirmed from the actual API rules.

Reconstructed State-Based Test Cases

Test Case 1: Initiate a valid payment. Verify the Pending response.

Test Case 2: Receive payment confirmation. Verify the transition from Pending to Success.

Test Case 3: Trigger an eligible reversal. Verify that the transaction reaches the specified Reversal state and that balances are reconciled correctly.

Test Case 4: Simulate a timeout before confirmation. Verify the Timeout response and subsequent reconciliation behavior.

Test Case 5: Retry a request after a timeout. Verify that the retry does not create a duplicate payment.

Test Case 6: Attempt a transition that is not permitted by the API specification, such as changing a terminal state directly back to Pending. Verify the defined rejection or recovery behavior.

Test Case 7: Submit duplicate status callbacks. Verify that transaction processing remains consistent.

Improved Testing Strategy

Use a state-transition table to list each current state, triggering event, expected next state, and expected response. Test every permitted transition and selected invalid transitions.

Validate API fields, transaction identifiers, status codes, timestamps, and idempotency behavior. Include failure and retry scenarios.

Conclusion

State Transition Testing is appropriate for payment gateways because the correctness of an operation depends on the transaction's current state and the event received. A reconstructed state model helps identify missing transitions, duplicate processing, and inconsistent payment results.

7. Reverse Engineering an Encrypted Communication Module Using Fuzzing and BVA

Introduction

An encrypted communication module exposes input parameters and encrypted outputs without revealing its internal implementation. Fuzzing and Boundary Value Analysis can be used to investigate input validation and error handling.

The objective is to determine whether malformed or extreme inputs are handled safely before encryption or transmission.

System Decomposition

The module can be divided into:

1. Input parsing.
2. Input validation.
3. Length and format checking.
4. Encryption processing.
5. Output encoding.
6. Error handling.
7. Communication or transmission.

Application of Fuzzing

Fuzzing supplies unexpected, malformed, or unusual inputs to identify crashes, incorrect responses, and validation failures.

For an authorized test environment, useful inputs include:

- Empty input.
- Missing fields.
- Incorrect data types.
- Malformed encoding.
- Very long strings.

- Invalid length fields.
- Unexpected special characters.
- Truncated data.
- Invalid key or parameter formats.

The expected result is safe rejection or controlled error handling, without crashes, memory corruption, or disclosure of sensitive information.

Application of Boundary Value Analysis

Assume the module accepts input lengths from a minimum L to maximum U. Test values immediately below, at, and above these limits.

For example, test $L - 1$, L, $L + 1$, $U - 1$, U, and $U + 1$, subject to the actual input specification.

Also test empty inputs, oversized messages, and encoded data near the maximum supported length.

Reconstructed Test Strategy

For every test, verify whether the input is accepted or rejected, whether the module produces the expected response, and whether it remains stable.

Inspect logs for accidental disclosure of keys, plaintext, credentials, or other sensitive values. Verify that validation occurs before unsafe processing and that output encoding is correct.

Fuzzing and BVA can expose conditions associated with buffer overflows or injection vulnerabilities, but they cannot independently prove that such vulnerabilities are absent.

Conclusion

Combining fuzzing with BVA systematically examines malformed inputs and length boundaries. This approach improves input validation and helps identify unsafe processing and error-handling defects in encrypted communication modules.

8. Reverse Engineering an Insurance Premium Calculator Using CFG and Path Testing

Introduction

An insurance premium calculator reports 100% statement coverage but fails to detect defects in multi-condition if statements. This indicates that the existing testing strategy did not adequately exercise decision outcomes or combinations of Boolean conditions.

Analysis of the Existing Approach

Statement coverage confirms that executable statements have been executed. However, a statement may execute even when some conditions within a decision have never been evaluated to both True and False.

For example, consider a premium rule:

IF age is within the permitted range
  AND driving record is acceptable
  AND vehicle type is eligible
THEN calculate the standard premium
ELSE apply the specified alternative rule

A test case in which all conditions are True may execute the premium calculation statement, but it does not test the False outcomes of the individual conditions.

System Decomposition

The insurance premium calculator can be divided into:

1. Customer input validation.
2. Age validation.
3. Driving-record evaluation.
4. Vehicle eligibility checking.
5. Risk-category selection.

6. Premium calculation.
7. Output generation and error handling.

Construction of a Conceptual CFG

Start
|
Validate Input
|
Is Input Valid?
| Yes | No
Evaluate Risk Display Error
|
Check Age
|
Check Driving Record
|
Check Vehicle Type
|
Calculate Premium or Apply Alternative Rule
|
Display Result
|
End

The actual CFG should be constructed from the real code, including every decision and alternative branch.

Identification of Independent Paths

For a decision involving three independent Boolean conditions, test combinations that cover:

- All conditions True.
- Age condition False.
- Driving-record condition False.
- Vehicle-eligibility condition False.

Additional tests should examine combinations in which multiple conditions are False, invalid inputs, and alternative premium rules.

The exact independent paths depend on the actual CFG and any nested decisions.

Reconstructed Path Testing Strategy

Apply branch coverage to execute every decision outcome. Use basis-path testing to select linearly independent execution paths based on the CFG's cyclomatic complexity.

Verify the premium amount, selected risk category, validation messages, and error-handling behavior for each test.

The question requests 100% branch and path coverage. Complete path coverage is feasible only for a finite, appropriately bounded graph without unrestricted loops. If loops or many combinations exist, document the limitations and use basis-path and risk-based testing.

Conclusion

Statement coverage alone is insufficient for complex insurance calculations. CFG reconstruction, branch coverage, and basis-path testing provide stronger evidence that important decision outcomes are tested and reduce the risk of incorrect premium calculations.

9. Reverse Engineering a Cloud File Synchronization Tool Using ECP, BVA, and Stress Testing

Introduction

A cloud-based file synchronization tool produces user activity logs and defect reports involving lost files. The reported problems are related to file size, file type, and network speed.

Reverse engineering can help infer the original equivalence classes and identify missing boundary and stress tests.

System Decomposition

The synchronization tool can be divided into:

1. File selection and metadata retrieval.
2. File-type validation.
3. File-size validation.
4. Network-condition detection.
5. Upload or download processing.
6. Synchronization conflict resolution.
7. Retry and recovery management.
8. File verification and logging.

Identification of Equivalence Classes

For file size, possible classes include empty files, small valid files, ordinary files, files near the maximum supported size, and files exceeding the limit.

For file type, classes may include supported file formats, unsupported formats, corrupted files, and files with misleading or missing extensions.

For network conditions, classes may include stable high-speed connections, slow connections, intermittent connectivity, and complete network failure.

These classes represent inferred categories and must be confirmed against the actual system requirements.

Application of Boundary Value Analysis

If the maximum supported file size is S, test files of sizes $S - 1$, S, and $S + 1$ bytes, where applicable.

Also test zero-byte files, very small files, files with large metadata, and files close to storage limits.

For network conditions, test transitions between connected, slow, interrupted, and restored connectivity.

Application of Stress Testing

Increase the number of simultaneous files, total data volume, and concurrent synchronization requests. Observe whether the system maintains correct file contents and metadata.

Interrupt the network during uploads and downloads, restart the application, and verify that incomplete operations resume or recover according to the specification.

Compare file hashes before and after synchronization to detect content corruption. Verify that retry operations do not create duplicate or conflicting files.

Reconstructed Testing Strategy

Combine ECP, BVA, stress testing, interruption testing, and synchronization-conflict testing. Review user activity logs and defect reports to identify conditions associated with file loss.

Conclusion

ECP organizes file and network inputs into meaningful classes, while BVA focuses on file-size limits. Stress and interruption testing expose problems under heavy workloads and unstable connections. Together, these techniques help prevent data loss and improve synchronization reliability.

10. Reverse Engineering an E-Commerce Search and Filter Module Using Decision Table Testing

Introduction

An e-commerce search and filter module generates SQL queries, but its source code and test documentation are unavailable. By examining query parameters and resulting datasets, the testing team can infer the filtering

logic and reconstruct a decision-table-based test strategy.

System Decomposition

The search and filter module can be divided into:

1. Search keyword processing.
2. Category filtering.
3. Price-range filtering.
4. Availability filtering.
5. Sorting and pagination.
6. Boolean condition evaluation.
7. SQL query generation.
8. Result retrieval and output display.

Reverse Engineering Search Logic

The SQL queries and returned datasets can reveal whether filters are combined using AND, OR, and NOT operators.

For example:

- AND returns records satisfying all specified conditions.
- OR returns records satisfying at least one of the specified conditions.
- NOT excludes records satisfying a specified condition.

Suppose a customer searches for products in the Electronics category with a price below a specified limit. The result set should contain only products satisfying both conditions if the intended rule uses AND.

If a search allows either of two categories, the category conditions may be combined using OR.

The actual logic must be inferred from the provided queries and observed results; the assessment does not include concrete query examples.

Decision Table Testing

A decision table should represent combinations of filter conditions and their expected outcomes.

Relevant conditions include:

- Keyword matches or does not match.
- Category matches or does not match.
- Price is within or outside the specified range.
- Product is available or unavailable.
- Optional filters are supplied or omitted.

Expected actions include returning matching products, returning an empty result, excluding unavailable products, or reporting invalid filter inputs.

Reconstructed Test Cases

Test Case 1: Search using a valid keyword with no additional filters. Verify that matching products are returned.

Test Case 2: Apply a category filter. Verify that all returned products belong to the selected category.

Test Case 3: Apply a minimum and maximum price filter. Verify that products outside the permitted range are excluded.

Test Case 4: Combine keyword and category using AND. Verify that results satisfy both conditions.

Test Case 5: Combine alternative categories using OR. Verify that results match at least one permitted category.

Test Case 6: Apply a NOT condition. Verify that the specified products or category are excluded.

Test Case 7: Use contradictory filters. Verify that the system returns an empty result or follows the specified validation rule.

Test Case 8: Submit missing, malformed, or invalid filter values. Verify that the application handles them safely.

Test Case 9: Test sorting and pagination together with filters. Verify that the returned pages contain only matching products without unintended duplication.

Evaluation of Completeness

The reconstructed decision table should cover each meaningful condition and its expected action. Pairwise testing can supplement the suite when many filters interact, but critical Boolean combinations should be tested explicitly.

SQL generation must also be checked for parameterization and safe handling of user inputs. Expected results should be compared with independently constructed datasets.

Conclusion

Decision Table Testing helps reconstruct complex search logic by connecting filter combinations with expected results. Combining decision tables, ECP, negative testing, and SQL safety checks improves test completeness and helps identify defects in e-commerce search functionality.